NESNE TABANLI PROGRAMLAMA

Laboratuvar Föyü

Hafta 7 — İsim Alanları, Takma İsimler ve Kütüphane (.dll)

Öğrenci Yönergesi — Adım Adım Uygulama

Bilgi	Değer
Ders	Nesne Tabanlı Programlama (2. Sınıf)
Konu	namespace · using · alias · Class Library (.dll)
Ortam	Visual Studio Community 2026 · .NET 10 (LTS)
Dil	C#
Süre	Yaklaşık 2–3 saat
Teslim	Pdf-doc

1. Bu Haftanın Hedefi

Bu hafta kodu isim alanlarıyla (namespace) düzenlemeyi, aynı isimli sınıfları using takma isim (alias) ile ayırmayı ve kendi kodumuzu bir kütüphane (.dll) hâline getirip başka bir projeden referansla kullanmayı öğreneceğiz.

2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- namespace tanımlayıp sınıfları mantıksal gruplara ayırmak,
- using ile isim alanı bildirmek ve using alias ile takma isim vermek,
- Aynı isimli iki sınıfı farklı isim alanlarında çakışmadan kullanmak,
- Bir Class Library (sınıf kütüphanesi) projesi oluşturup .dll üretmek,
- Kütüphaneyi başka bir projeye referans ekleyip using ile kullanmak.

3. Temel Kavramlar

3.1. İsim Alanı (namespace)

Sınıfları, yapıları ve enum'ları mantıksal olarak gruplayan yapıdır; isim çakışmalarını önler. namespace anahtar sözcüğüyle tanımlanır ve bir blok (parantez) içine türler yazılır. Bir isim alanındaki türe dışarıdan nokta ile erişilir: IsimAlani.Sinif.

3.2. using ve Takma İsim (alias)

using IsimAlani; satırı o isim alanındaki türleri tam ad yazmadan kullanmayı sağlar. İki farklı isim alanında aynı adlı sınıf varsa, hangisinin kastedildiği belirsizleşir; bunu takma isim (alias) ile çözeriz:

```
using Sinif1 = IsimAlani1.Sinif;   // takma isim
using yaz = System.Console;         // hazır sınıflara da alias verilebilir
```

3.3. Kütüphane Dosyası (.dll)

Bir Class Library projesi derlendiğinde çalıştırılabilir bir .exe değil, başka projelerin kullanabileceği bir .dll üretir. İçindeki türler başka projeden görülebilmesi için public olmalıdır. Kütüphane, kullanılacağı projeye referans olarak eklenir.

4. Uygulama — Bölüm A: İsim Alanları ve Takma İsim

Aynı adı (Hesap) taşıyan iki sınıfı iki farklı isim alanında tanımlayıp, takma isimlerle ayırarak kullanacağız.

Önce **Hafta7_IsimAlanlari** adında bir Konsol Uygulaması (C#, .NET 10) oluşturun. Program.cs dosyasını aşağıdaki gibi yazın:

```
using System;
using BankaHesap = Bankacilik.Hesap;   // takma isim 1
using OyunHesap  = Oyun.Hesap;          // takma isim 2

namespace Bankacilik
{
    class Hesap
    {
        public double Bakiye;
        public void Yaz() => Console.WriteLine($"Banka bakiyesi: {Bakiye} TL");
    }
}

namespace Oyun
{
    class Hesap
    {
        public int Puan;
        public void Yaz() => Console.WriteLine($"Oyun puanı: {Puan}");
    }
}

namespace Hafta7_IsimAlanlari
{
    class Program
    {
        static void Main(string[] args)
        {
            BankaHesap banka = new BankaHesap();
            banka.Bakiye = 1500;
            banka.Yaz();

            OyunHesap oyun = new OyunHesap();
            oyun.Puan = 350;
            oyun.Yaz();
        }
    }
}
```

Ne görmelisiniz?
Banka bakiyesi: 1500 TL
Oyun puanı: 350

⚠Takma isim neden gerekli?
İki isim alanında da Hesap adlı sınıf var. Doğrudan "Hesap" yazsaydınız derleyici hangisini kastettiğinizi bilemez ve hata verirdi.
Takma isim (BankaHesap, OyunHesap) bu belirsizliği ortadan kaldırır.

5. Uygulama — Bölüm B: Kütüphane (.dll) Oluşturma ve Kullanma

Kendi matematik kütüphanemizi ayrı bir projede yazıp, ana projeden referansla kullanacağız.

Adım 1 — Sınıf kütüphanesi projesi ekleyin

1. Visual Studio'da aynı çözüme (solution) yeni bir proje ekleyin: Çözüm'e sağ tık → Ekle → Yeni Proje (Add → New Project).
2. Şablonlardan Sınıf Kitaplığı (Class Library / C#) seçin, ad olarak MatematikKutuphane yazın ve .NET 10 ile oluşturun.
3. Oluşan Class1.cs dosyasını Islemler.cs olarak yeniden adlandırın (veya yeni sınıf ekleyin).

Islemler.cs içeriği (türün public olmasına dikkat edin — başka projeden görülecek):

```
namespace MatematikKutuphane
{
    public class Islemler
    {
        public static int Topla(int a, int b)
        {
            return a + b;
        }

        public static long Faktoriyel(int n)
        {
            long sonuc = 1;
            for (int i = 2; i <= n; i++)
                sonuc *= i;
            return sonuc;
        }
    }
}
```

Adım 2 — Kütüphaneyi ana projeye referans ekleyin

4. Ana konsol projenizin (ör. Hafta7_IsimAlanlari veya yeni bir konsol projesi) altındaki Bağımlılıklar (Dependencies) öğesine sağ tık → Proje Başvurusu Ekle (Add Project Reference).
5. Açılan listede MatematikKutuphane projesini işaretleyip Tamam deyin.

csc ile alternatif (bilgi)
Komut satırından da kütüphane üretilebilir: csc /t:library Islemler.cs → Islemler.dll
Sonra ana kodu derlerken referans verilir: csc /r:Islemler.dll Program.cs
Derste Visual Studio'nun Proje Başvurusu yöntemini kullanacağız; csc yöntemi yalnızca bilgi içindir.

Adım 3 — Kütüphaneyi kullanın

Ana projenin Program.cs dosyasında kütüphaneyi using ile dâhil edip metotlarını çağırın:

```
using System;
using MatematikKutuphane;     // kütüphaneyi dâhil et

namespace AnaProje
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine($"Topla(7, 5) = {Islemler.Topla(7, 5)}");
            Console.WriteLine($"5! = {Islemler.Faktoriyel(5)}");
        }
    }
}
```

Ne görmelisiniz?
Topla(7, 5) = 12

```
5! = 120
```

6. Kendi Göreviniz (Lab Teslimi)

Uygulamayı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

1. Bölüm A'ya üçüncü bir isim alanı ekleyin (ör. Okul) ve içinde yine Hesap adlı (öğrenci not hesabı) bir sınıf tanımlayın; takma isimle (OkulHesap) Main'de kullanın.
2. MatematikKutuphane'ye bir metot daha ekleyin: public static int Us(int taban, int us) (tabanın üssünü hesaplasın) ve ana projede çağırın.
3. Kütüphanenin içine iç içe bir isim alanı (ör. MatematikKutuphane.Ileri) tanımlayıp bir sınıf koyun; ana projeden tam adıyla (MatematikKutuphane.Ileri.SinifAdi) erişin.
4. Tüm sonuçları ekrana düzenli biçimde yazdırın.

7. Sık Yapılan Hatalar

- **Kütüphane türünü public yapmamak:** Farklı projeden erişilecek sınıf ve metotlar public olmalı; değilse 'erişilemez' hatası alırsınız.
- **Referans eklemeyi unutmak:** using MatematikKutuphane; yazmanız yetmez; önce projeye referans eklenmelidir.
- **Alias'ı yanlış tanımlamak:** using Ad = IsimAlani.Sinif; biçiminde, tam yol ile yazılmalıdır.
- **Aynı adlı sınıfı alias'sız kullanmak:** İki isim alanında aynı ad varsa doğrudan kullanım belirsizlik hatası verir.
- **Başlangıç projesini ayarlamamak:** Class Library çalıştırılamaz; başlangıç projesi konsol projesi olmalıdır.

8. Değerlendirme

Ölçüt	Puan
Çözüm derleniyor ve çalışıyor	15
İki isim alanı + aynı adlı sınıflar	15
using takma isim (alias) doğru kullanılmış	15
Class Library projesi oluşturulmuş (public tür)	20
Referans eklenmiş ve kütüphane kullanılmış	20
Görev: Us metodu + iç içe isim alanı	15
TOPLAM	100

9. Ödev (Gelecek Haftaya Hazırlık)

MatematikKutuphane'yi büyütün: bir sayının asal olup olmadığını döndüren public static bool AsalMi(int n) metodu ekleyin ve ana projede 1–20 arası sayıların asallık durumunu yazdırın. Kütüphaneleri büyüttükçe, türleri alt isim alanlarına (iç içe namespace) bölmek kodu daha düzenli tutar; bunu uygulamaya çalışın.